AI Assisted Coding

Lab Assignment – 11.1

Name : P. Gouri Prasad Varma

Hall.t.No. : 2303A51035

Batch : 01

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.

2. Event Registration System – Manage participants in events with quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

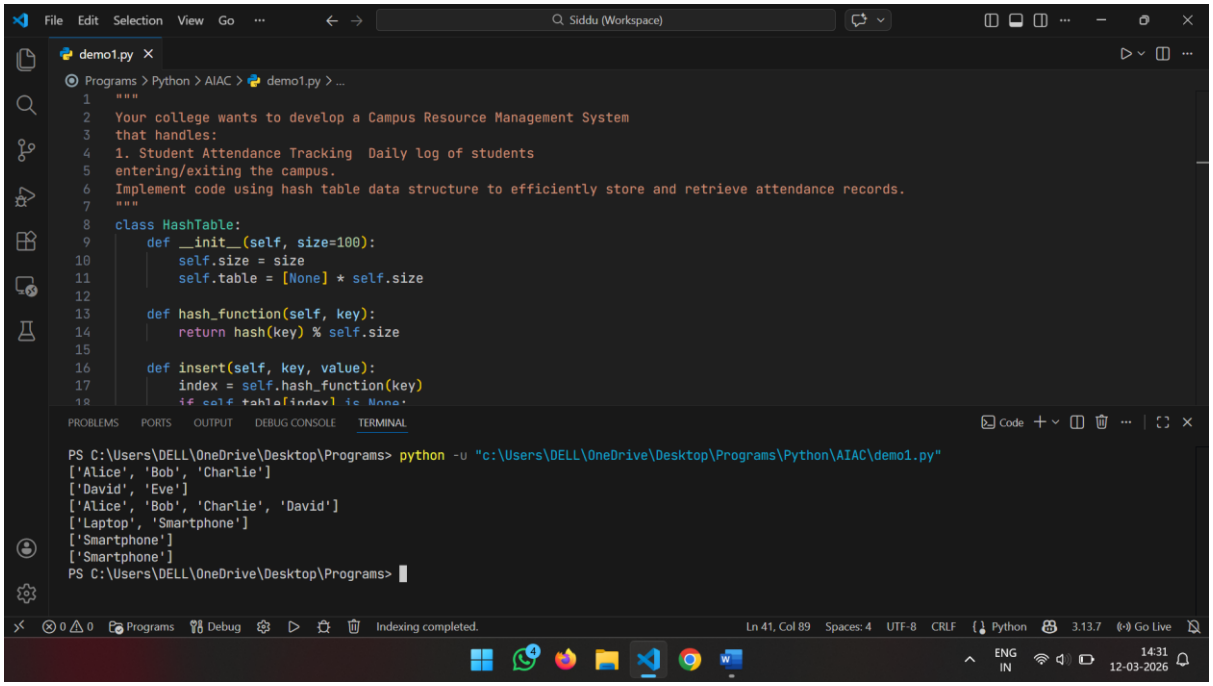
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Attendance Tracking	**Hash Table**	It provides $O(1)$ average time complexity for lookups and updates. Using a Student ID as a key allows for near-instant logging of entry and exit times.
Event Registration	**Binary Search Tree (BST)**	A BST (or a balanced version like an AVL tree) keeps participant names sorted automatically. This allows for quick alphabetical searches and efficient removals if someone cancels.
Library Borrowing	**Hash Table**	Searching for a book by its ISBN or unique ID needs to be instantaneous. A Hash Table maps the ID directly to the book's status and due date without scanning the whole collection.
Bus Scheduling	**Graph**	Routes are essentially a series of nodes (stops) and edges (paths). A Graph allows the system to calculate the shortest path or find all stops connected to a specific route.
Cafeteria Queue	**Queue**	This follows the **FIFO (First-In, First-Out)** principle. The first student to line up should be the first one served, ensuring fairness and order in the dining hall.

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

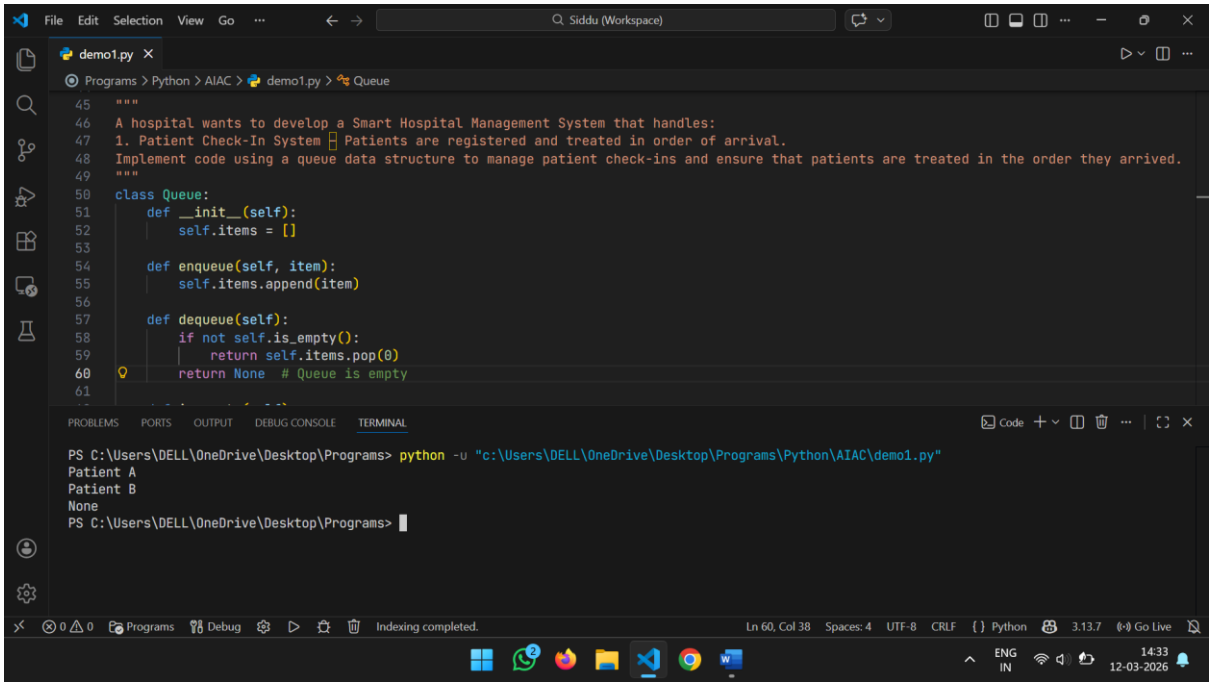
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Patient Check-In	**Queue**	Standard check-ins follow a **FIFO (First-In, First-Out)** rule. This ensures that non-emergency patients are seen in the exact order they arrived at the desk.
Emergency Handling	**Priority Queue**	In an ER, the "next" person isn't the one who arrived first, but the one with the highest medical urgency. A Priority Queue allows the system to always serve the most critical case first, regardless of arrival time.
Medical Records	**Hash Table**	When a doctor needs a patient's history, they need it instantly. Mapping a Patient ID to a record in a Hash Table provides $O(1)$ average time complexity for retrieval.
Appt. Scheduling	**BST (or AVL Tree)**	Appointments need to be kept in chronological order. A Binary Search Tree keeps them sorted by time, making it easy to find "the next available slot" or print a day's schedule in order.
Room Navigation	**Graph**	A hospital is a network of wards, floors, and corridors. A Graph represents these locations as nodes and the hallways as edges, allowing for "shortest path" navigation (e.g., from the ER to the ICU).

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

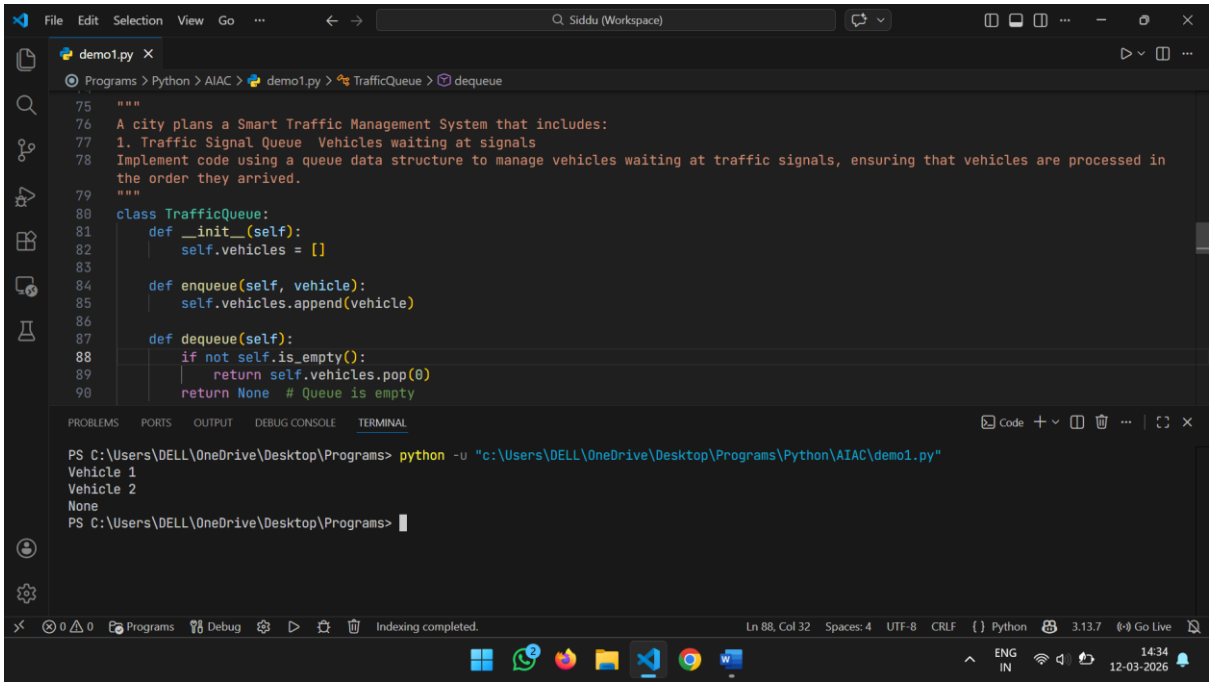
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Traffic Signal Queue	**Queue**	This follows the **FIFO (First-In, First-Out)** principle. The first vehicle to reach the red light should be the first to move when it turns green to maintain a fair and natural traffic flow.
Emergency Priority	**Priority Queue**	Ambulances and fire trucks cannot wait in a standard line. A Priority Queue ensures these vehicles are bumped to the front of the "processing" list regardless of when they arrived at the intersection.
Vehicle Lookup	**Hash Table**	Police or automated cameras need to link a license plate to a driver instantly. A Hash Table provides $O(1)$ average time complexity, allowing for near-instant retrieval among millions of records.
Road Network	**Graph**	A city is a collection of intersections (nodes) and roads (edges). A Graph is the only way to logically map these connections and run algorithms like Dijkstra's to find the fastest route across town.
Parking Slots	**Hash Table**	To track whether slot "A-101" is occupied, a Hash Table (or a simple Bit Map/Array if IDs are contiguous) allows for instant

Feature	Data Structure	Justification
		status checks and updates without searching through the whole lot.

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

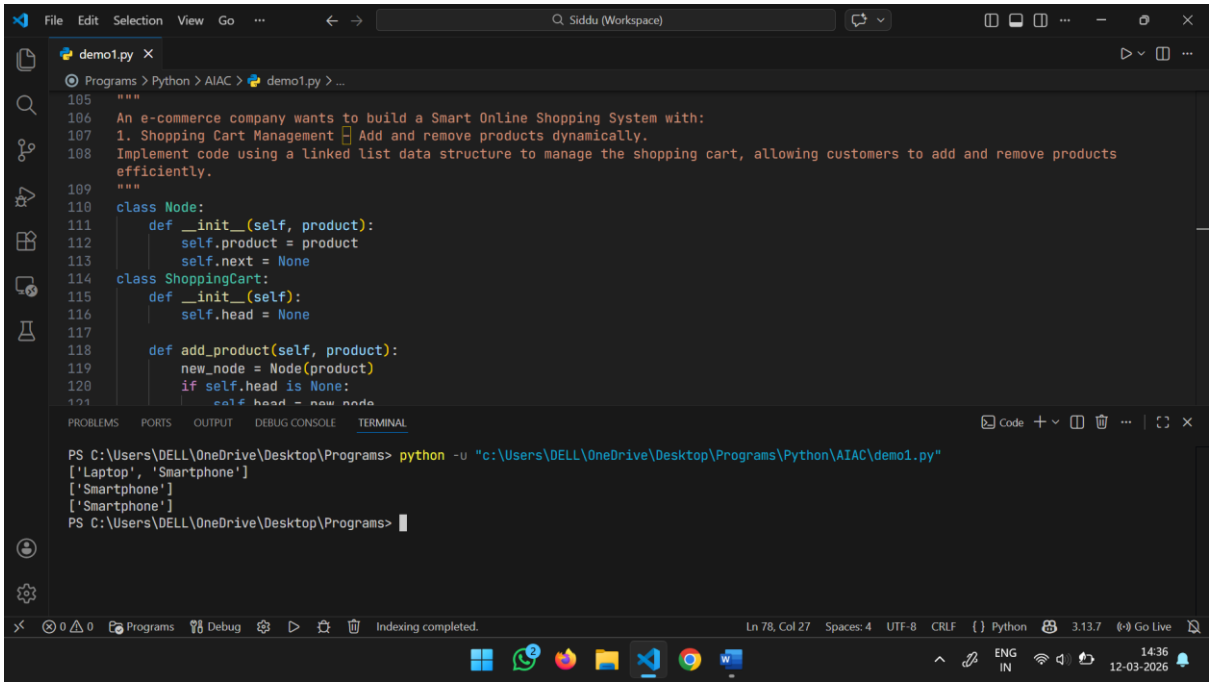
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Shopping Cart	**Linked List**	A Doubly Linked List allows for $O(1)$ additions and removals of products at any point. This is ideal for a dynamic cart where users might remove an item from the middle of their list without re-indexing the entire collection.
Order Processing	**Queue**	Orders must be handled using the **FIFO (First-In, First-Out)** principle to ensure fairness. The first customer to click "Checkout" is the first whose order is sent to the warehouse for packing.
Top-Selling Tracker	**Priority Queue**	A Max-Priority Queue (often implemented as a Heap) keeps the products with the highest sales counts at the top. This allows the platform to instantly retrieve the "Top N" trending products for the homepage.
Search Engine	**Hash Table**	When a user enters a specific Product ID or SKU, the system needs to find that product instantly among millions. A Hash Table provides $O(1)$ average time complexity for these direct lookups.
Delivery Routes	**Graph**	Warehouses and delivery addresses are nodes, while roads are edges. A Graph structure is necessary to calculate the most

Feature	Data Structure	Justification
		efficient path for drivers, often using algorithms like Dijkstra's or A*.

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing

2. Urgent Shipment Priority

3. Product Lookup by Code

4. Warehouse Network Connectivity

5. Inventory Add/Remove Operations

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

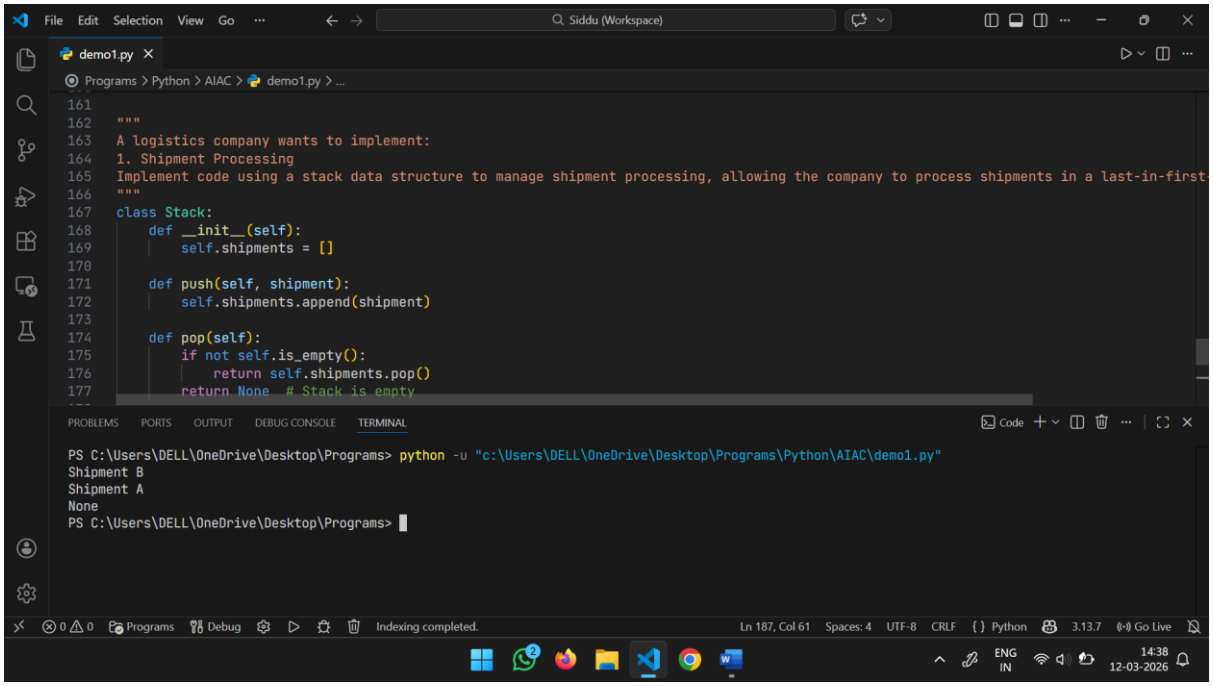
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Shipment Processing	**Queue**	Standard shipments are processed using the **FIFO (First-In, First-Out)** principle. This ensures that the first package to arrive at the sorting facility is the first one loaded onto a truck.
Urgent Shipment Priority	**Priority Queue**	"Express" or "Overnight" packages must jump the line. A Priority Queue allows the system to always extract the highest-priority item next, regardless of when it entered the system.
Product Lookup by Code	**Hash Table**	Warehouses contain millions of SKUs. A Hash Table allows a worker to scan a barcode and retrieve the product's aisle, shelf, and quantity in $O(1)$ average time.
Warehouse Network	**Graph**	Logistics hubs and distribution centers are nodes connected by transport routes (edges). A Graph is essential for calculating the most cost-effective path between two geographical points.
Inventory Operations	**Linked List**	For a dynamic inventory where products are frequently inserted or removed from various points in a list (not just the ends), a Linked List offers flexible $O(1)$ pointer updates without shifting elements.

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.

2. Loan Approval Priority Handling – High-value or urgent loans processed first.

3. Account Information Lookup – Fast retrieval of customer details using Account Number.

4. Transaction History Management – Maintain chronological transaction records.

5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

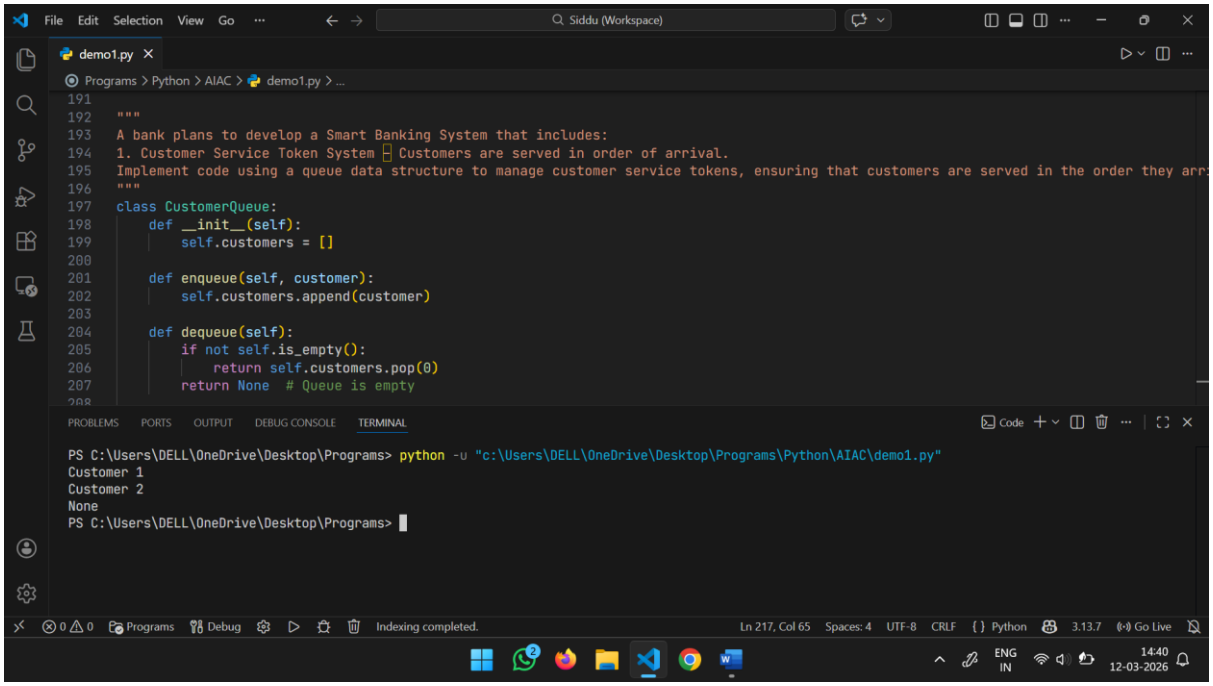
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Customer Token System	**Queue**	This follows the **FIFO (First-In, First-Out)** principle. It ensures fair treatment by serving the customer who took the first token before those who arrived later.
Loan Approval Priority	**Priority Queue**	Not all loans have the same urgency; a high-value corporate loan or an emergency credit line may need faster processing. A Priority Queue allows the bank to "jump" these cases to the front of the review pile.
Account Lookup	**Hash Table**	A bank manages millions of accounts. Using the Account Number as a key in a Hash Table allows for $O(1)$ average time complexity, retrieving details instantly without searching the entire database.
Transaction History	**Linked List**	Transactions are sequential and constantly growing. A Linked List (specifically a Doubly Linked List) allows the bank to easily append new transactions or traverse through history chronologically without needing contiguous memory.
ATM Network	**Graph**	ATMs are physical locations (nodes) connected by maintenance routes or data links (edges). A Graph helps the bank optimize

Feature	Data Structure	Justification
		cash replenishment routes or analyze the connectivity between different city branches.

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.

2. Assignment Submission Queue – Submissions processed in order of upload.

3. Top Performer Ranking – Rank students based on scores.

4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from the given list.

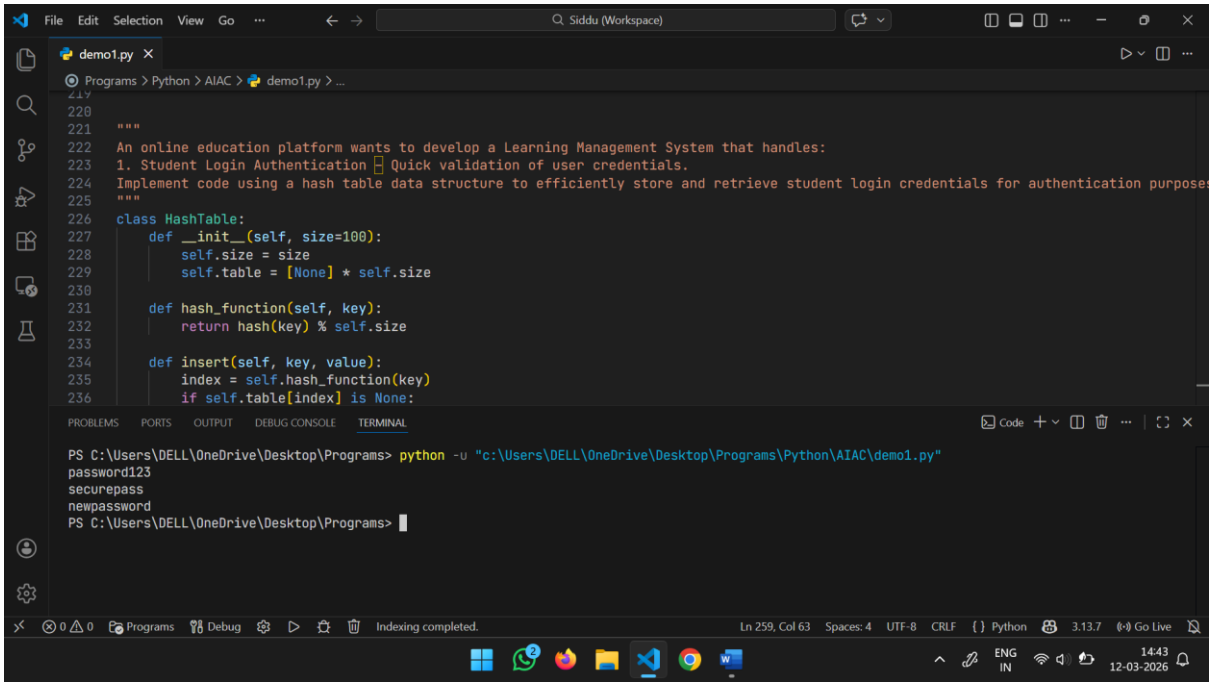
• Justify your selection in 2–3 sentences per feature.

• Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

• Feature → Data Structure → Justification table.

• Functional Python program with proper documentation.



Feature	Data Structure	Justification
Login Authentication	**Hash Table**	Student credentials (usernames/hashed passwords) need to be validated instantly. A Hash Table provides $O(1)$ average time complexity, ensuring that login speed remains constant even as the student body grows.
Assignment Submission	**Queue**	This follows the **FIFO (First-In, First-Out)** principle. It ensures that the grading system or the storage processor handles assignments in the exact order they were uploaded by students.
Top Performer Ranking	**Priority Queue**	Ranking students by scores is most efficient with a Max-Heap. A Priority Queue allows the platform to instantly extract the highest-scoring students for a leaderboard without sorting the entire database.
Course Prerequisites	**Graph**	Courses and their dependencies form a complex network. A Directed Acyclic Graph (DAG) is perfect for representing that Course A must be completed before Course B, allowing for topological sorting.
Undo Feature (Editor)	**Stack**	This relies on the **LIFO (Last-In, First-Out)** principle. Every action is "pushed" onto the stack; when a student hits

Feature	Data Structure	Justification
		"Undo," the most recent action is "popped" off to revert the state.

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.

2. Emergency Landing Priority – Aircraft in emergency get priority.

3. Flight Information Lookup – Fast search using Flight ID.

4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

• Choose suitable data structures from the provided list.

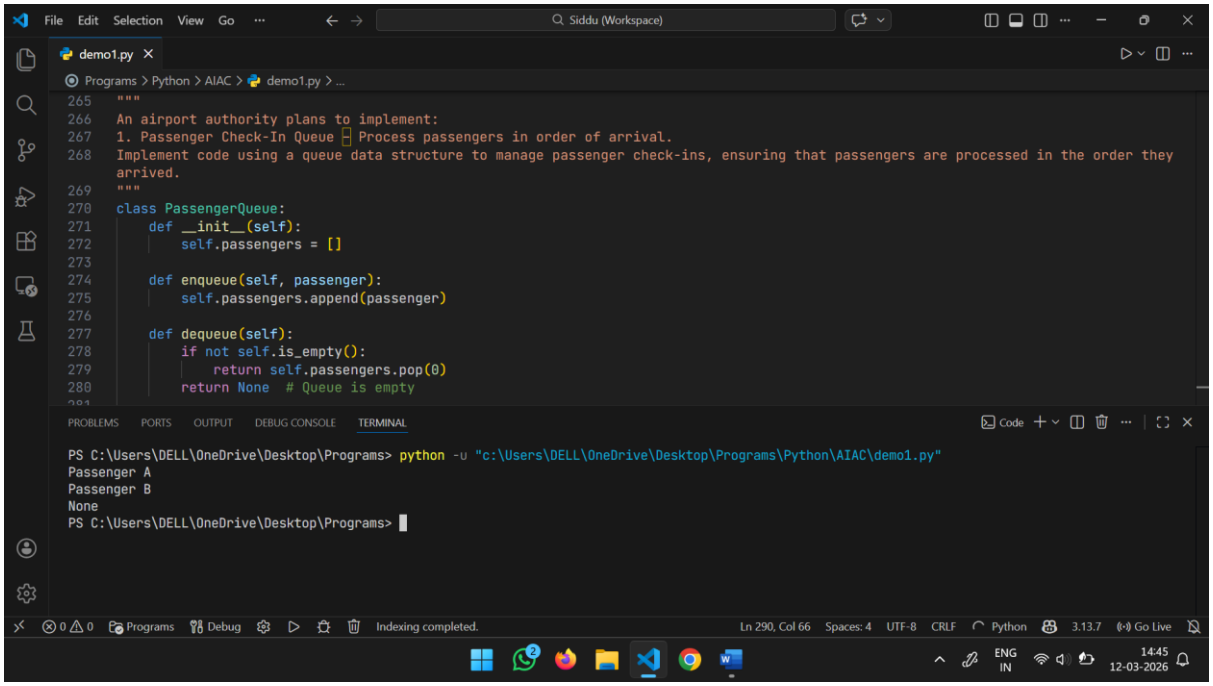
• Provide 2–3 sentence justification per feature.

• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.

• Working Python implementation.



Feature	Data Structure	Justification
Passenger Check-In	**Queue**	Check-in counters operate on a **FIFO (First-In, First-Out)** basis. This ensures that the first passenger to join the line is the first one assisted, maintaining order and fairness during peak travel times.
Emergency Landing	**Priority Queue**	In aviation, safety overrides arrival order. A Priority Queue allows the Air Traffic Control system to automatically move aircraft with emergency codes (e.g., low fuel, engine failure) to the top of the landing list.
Flight Lookup	**Hash Table**	Travelers and staff need to find flight status instantly among hundreds of daily departures. A Hash Table using the Flight ID as a key provides $O(1)$ average time complexity for these searches.
Air Route Connectivity	**Graph**	Global aviation is a giant network. Airports are nodes and flight paths are edges; a Graph is necessary to calculate connecting flights, total travel distance, and fuel requirements.
Boarding (Luggage)	**Stack**	Overhead bins and narrow cabin aisles often function on a **LIFO (Last-In, First-Out)** basis. The last bag pushed into a

Feature	Data Structure	Justification
		deep storage compartment or the last person in a narrow jet bridge is often the first one to come out.

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

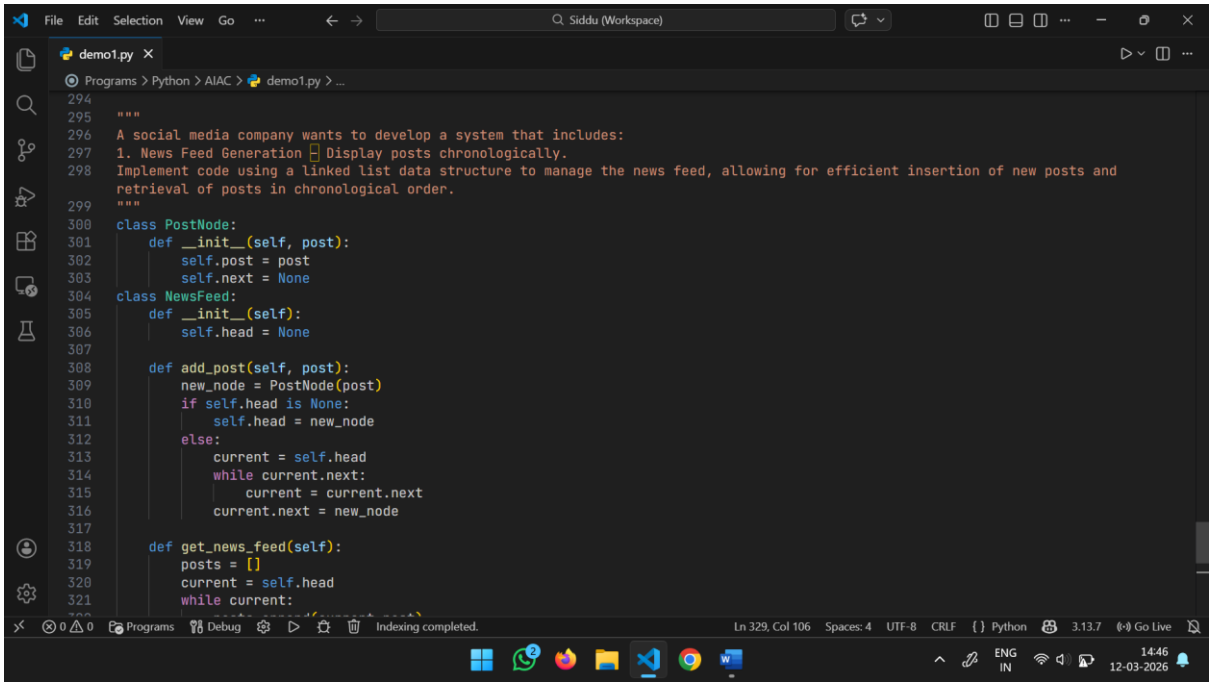
1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments



Feature	Data Structure	Justification
News Feed	**Linked List**	A Doubly Linked List is excellent for chronologically ordered data. It allows for $O(1)$ insertions at the head (newest post) and efficient traversal to load "older" posts as the user scrolls down.
Trending Hashtags	**Priority Queue**	To find what's "trending," the system needs to constantly track the most frequently used hashtags. A Max-Heap (Priority Queue) allows the top N hashtags to be extracted instantly without sorting the entire dataset.
Profile Search	**Hash Table**	Usernames are unique identifiers. Mapping a username to a profile object in a Hash Table ensures that searching for a friend happens in $O(1)$ average time, regardless of how many millions of users exist.
Friend Network	**Graph**	Social connections are non-linear and multidimensional. A Graph (using an Adjacency List) perfectly represents users as nodes and friendships as edges, enabling features like "Suggested Friends" or "Degrees of Separation."
Message Undo	**Stack**	This follows the **LIFO (Last-In, First-Out)** principle. By pushing sent messages onto a Stack, the system can easily "pop" the most

Feature	Data Structure	Justification
		recent message to remove it if the user hits the undo/withdraw button.

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment

2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

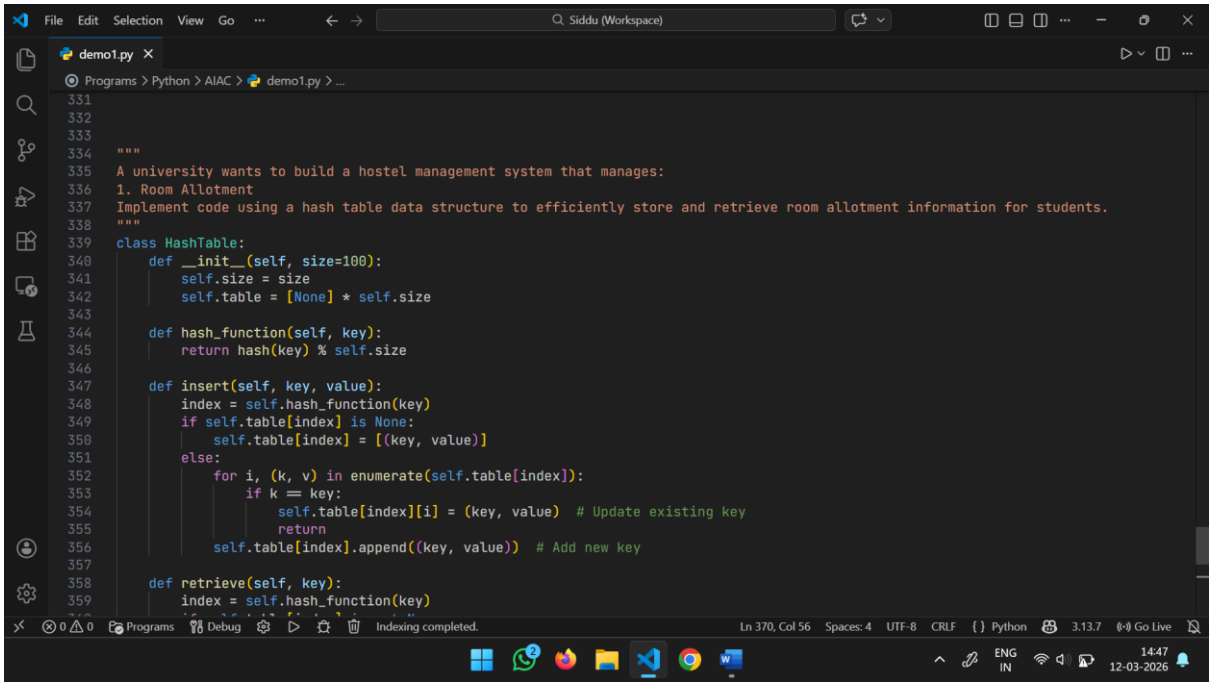
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings



Feature	Data Structure	Justification
Room Allotment	**Queue**	Rooms are generally allotted on a "First-Come, First-Served" basis during the registration period. A Queue ensures that the student who applied first gets the first choice of available rooms.
Medical Emergency	**Priority Queue**	In a hostel, health crises require immediate attention. A Priority Queue allows the system to process medical alerts based on severity (e.g., an unconscious student vs. a minor fever) rather than arrival time.
Student Record Retrieval	**Hash Table**	Retrieving a student's profile, contact info, or fee status by their Roll Number must be near-instant. A Hash Table provides $O(1)$ average time complexity, making lookups efficient regardless of hostel size.
Hostel Block Connectivity	**Graph**	A large campus has multiple blocks, messes, and security gates connected by pathways. A Graph represents these locations as nodes and paths as edges, helping in planning security patrols or maintenance routes.
Visitor Entry Log	**Linked List**	Visitors are logged sequentially as they enter and leave. A Linked List (or a Doubly Linked List) allows for efficient

Feature	Data Structure	Justification
		appending of new entries and is flexible for time-based tracking without requiring a fixed memory block.

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling

2. Fault Priority Handling

3. Consumer Record Lookup

4. Power Grid Connectivity

5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

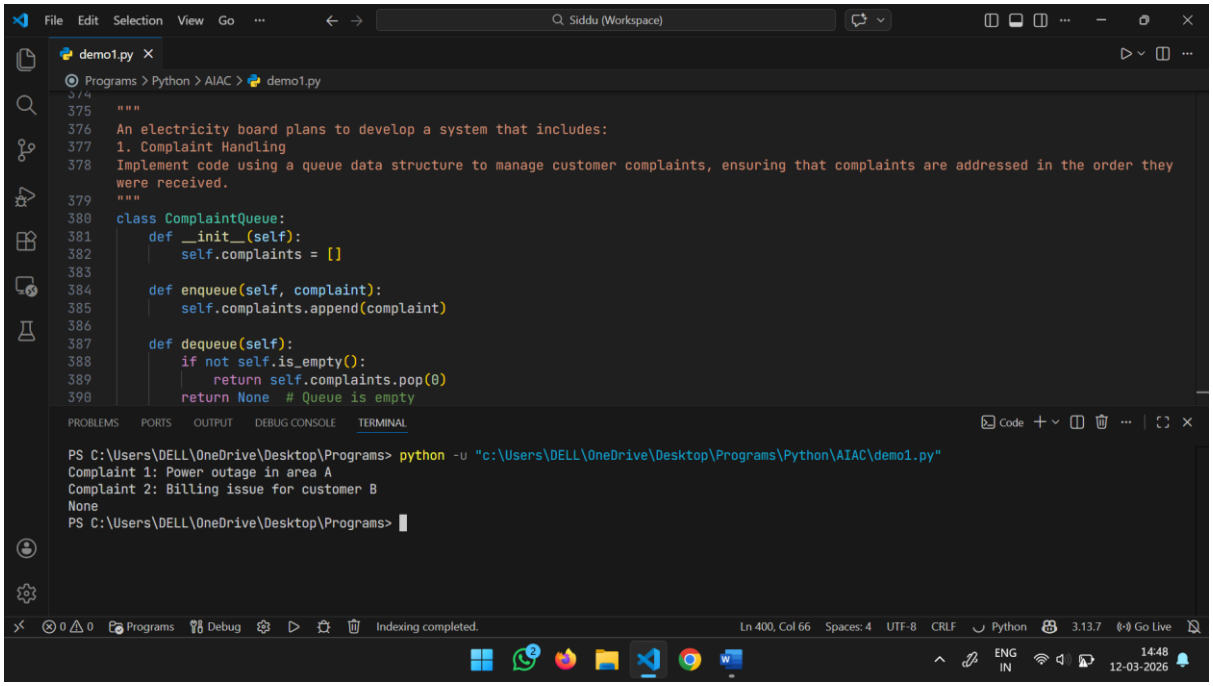
• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.



Feature	Data Structure	Justification
Complaint Handling	**Queue**	General complaints (like billing queries) should be addressed in the order they were received. A Queue follows the **FIFO (First-In, First-Out)** principle, ensuring fair and organized customer service.
Fault Priority Handling	**Priority Queue**	Not all power outages are equal; a total blackout at a hospital takes precedence over a flickering streetlamp. A Priority Queue allows the system to dispatch repair crews to critical faults first.
Consumer Record Lookup	**Hash Table**	With millions of consumers, finding details by a Consumer ID must be instantaneous. A Hash Table provides $O(1)$ average time complexity for retrieval, making it the fastest choice for direct lookups.
Power Grid Connectivity	**Graph**	The grid consists of power plants, substations, and transformers (nodes) connected by transmission lines (edges). A Graph is necessary to model flow, identify points of failure, and reroute power during maintenance.
Meter Reading History	**Linked List**	Meter readings are sequential data points recorded over time. A Linked List (or a Doubly Linked List) allows for efficient

Feature	Data Structure	Justification
		appending of new monthly readings and chronological traversal of usage history.